

C# 基礎課程 06

Bill Chung
V2026.1 台北夏季班

請尊重講師的著作權及智慧財產權!

Build School 課程之教材、程式碼等、僅供課程中學習用、請不要任意自行散佈、重製、分享，謝謝

靜態類別 static class

靜態類別

- 只包含靜態成員
- 無法產生實體
- 一定是密封的，無法被繼承
- 基底類別只能是 `Object Type`
- 沒有執行個體建構函式
- 如果要一個類別只有靜態成員，設計成靜態類別是不錯的想法
- 我們最常用靜態類別的情境在建立擴充方法的時候

設計靜態類別的常見情境

- 封裝並簡化常用功能
- 集中常數定義
- 擴充方法

LAB

靜態類別

新增一個方案及專案

- 方案名稱：StaticSamples
- 專案名稱：StaticSample001
- 範本：Console Application

建立靜態類別

```
public static class StringHelper
{
    public static string ToUpperTitleCase(string input)
    {
        if (string.IsNullOrEmpty(input))
        {
            return input;
        }
        return string.Join(' ', input.Split(' ', StringSplitOptions.RemoveEmptyEntries)
            .Select(word =>
                char.ToUpper(word[0]) + word.Substring(1).ToLower()));
    }
    public static string LeftOf(string source, string separator)
    {
        if (string.IsNullOrEmpty(source) || separator is null)
        {
            return source;
        }
        int index = source.IndexOf(separator, StringComparison.Ordinal);
        return index < 0 ? source : source.Substring(0, index);
    }
    public static string RightOf(string source, string separator)
    {
        if (string.IsNullOrEmpty(source) || separator is null)
        {
            return source;
        }

        int index = source.IndexOf(separator, StringComparison.Ordinal);
        return index < 0 ? source : source.Substring(index + separator.Length);
    }
}
```


修改 Main method

```
static void Main(string[] args)
{
    var source = "heLlo woRld froM sTatic meTHod";
    var result = StringHelper.ToUpperCase(source);
    Console.WriteLine(result);
    var left = StringHelper.LeftOf("first-middle-last", "-");
    Console.WriteLine(left);
    var right = StringHelper.RightOf("first-middle-last", "-");
    Console.WriteLine(right);
}
```



執行

擴充方法 Extension method

擴充方法

- 擴充方法讓您能將方法「加入」至現有類型，而不需要建立新的衍生類型 (Derived Type)、重新編譯，或是修改原始類型。擴充方法是一種特殊的靜態方法，但是需將它們當成擴充類型上的執行個體方法 (Instance Method) 來呼叫。
- 白話文就是擴充方法的設計是為了讓靜態方法可以假裝像是執行個體方法那樣使用。

擴充方法

- 要使用擴充方法的假裝技能，一定要引用該擴充方法的命名空間 (**namespace**)。
- 既然它本身是個靜態方法，所以一定可以用靜態的方式呼叫。
- 我們之前學習的 `linq` 語法都是 `Enumerable` 類別中的擴充方法（同時也是靜態方法）

討論

- 來點小實驗
- 試著把之前我們寫過的
linq 範例中的
`using System.Linq;` 拿掉，看看會發生甚麼事？



實作擴充方法 (Extension method) 的方式

Class一定是靜態的 (static)

```
public static class MyClass  
{  
    public static List<string> DoWhere(this List<string> source, Func<string, bool> pr  
edicate)
```

Method
也一定是靜態的 (static)

第一個參數要加上 **this** 關鍵字
(這就是為什麼可以假裝
是執行個體方法的關鍵)

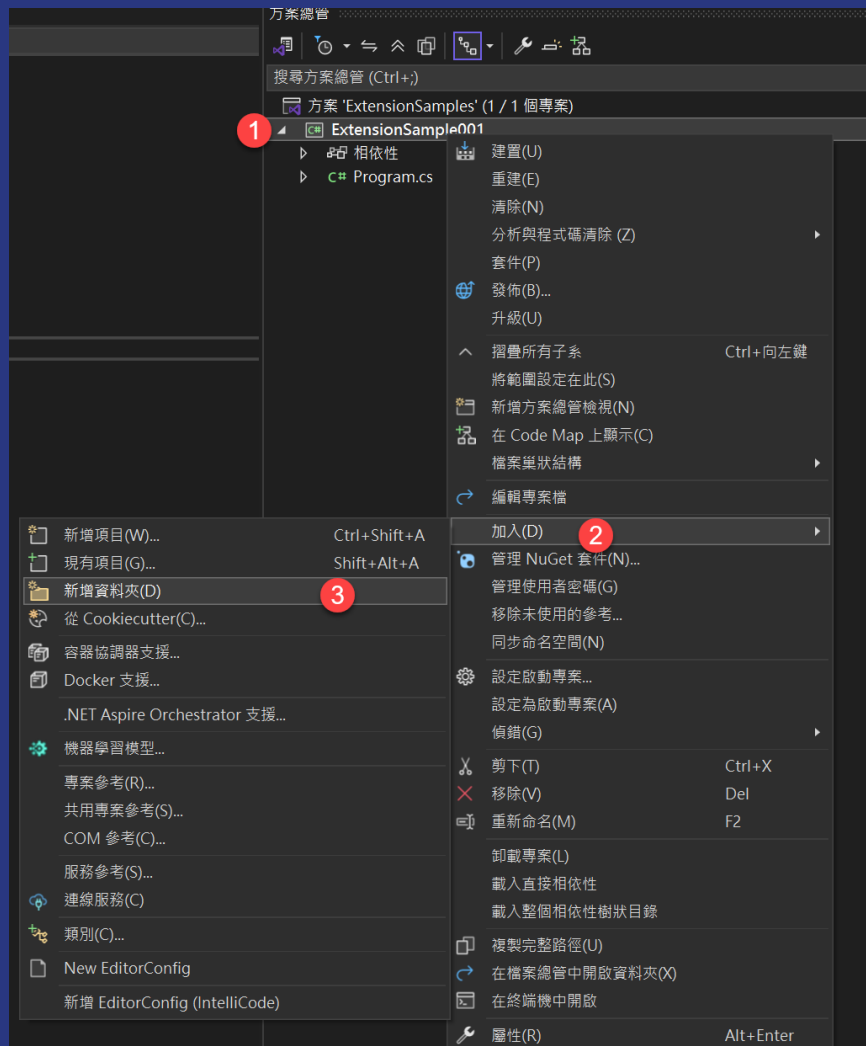
LAB

把 DelegateSample005 改成
Extension Method (擴充方法)

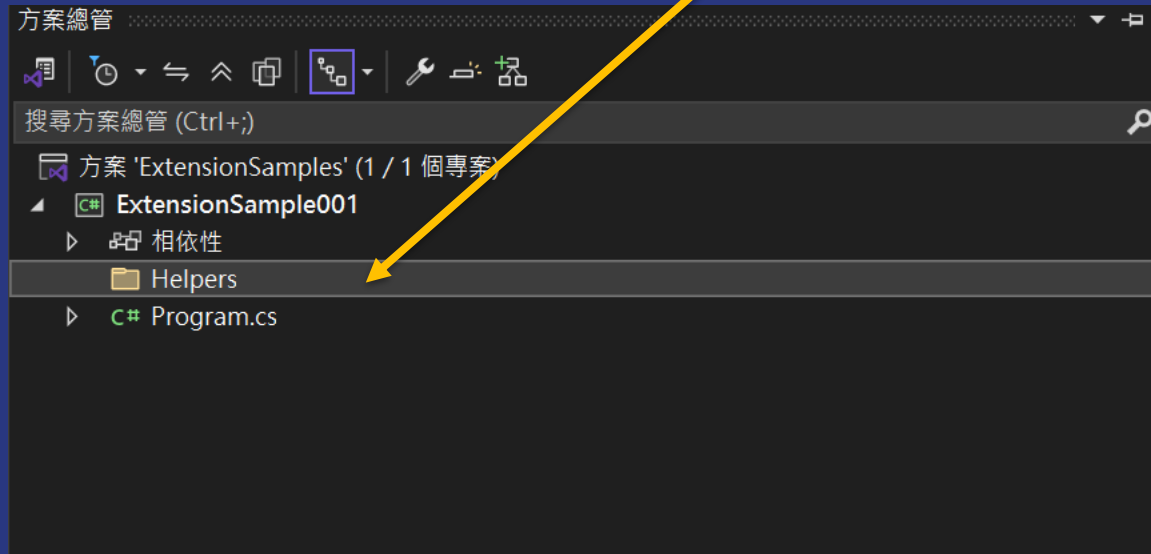
新增一個方案及專案

- 方案名稱：ExtensionSamples
- 專案名稱：ExtensionSample001
- 範本：Console Application

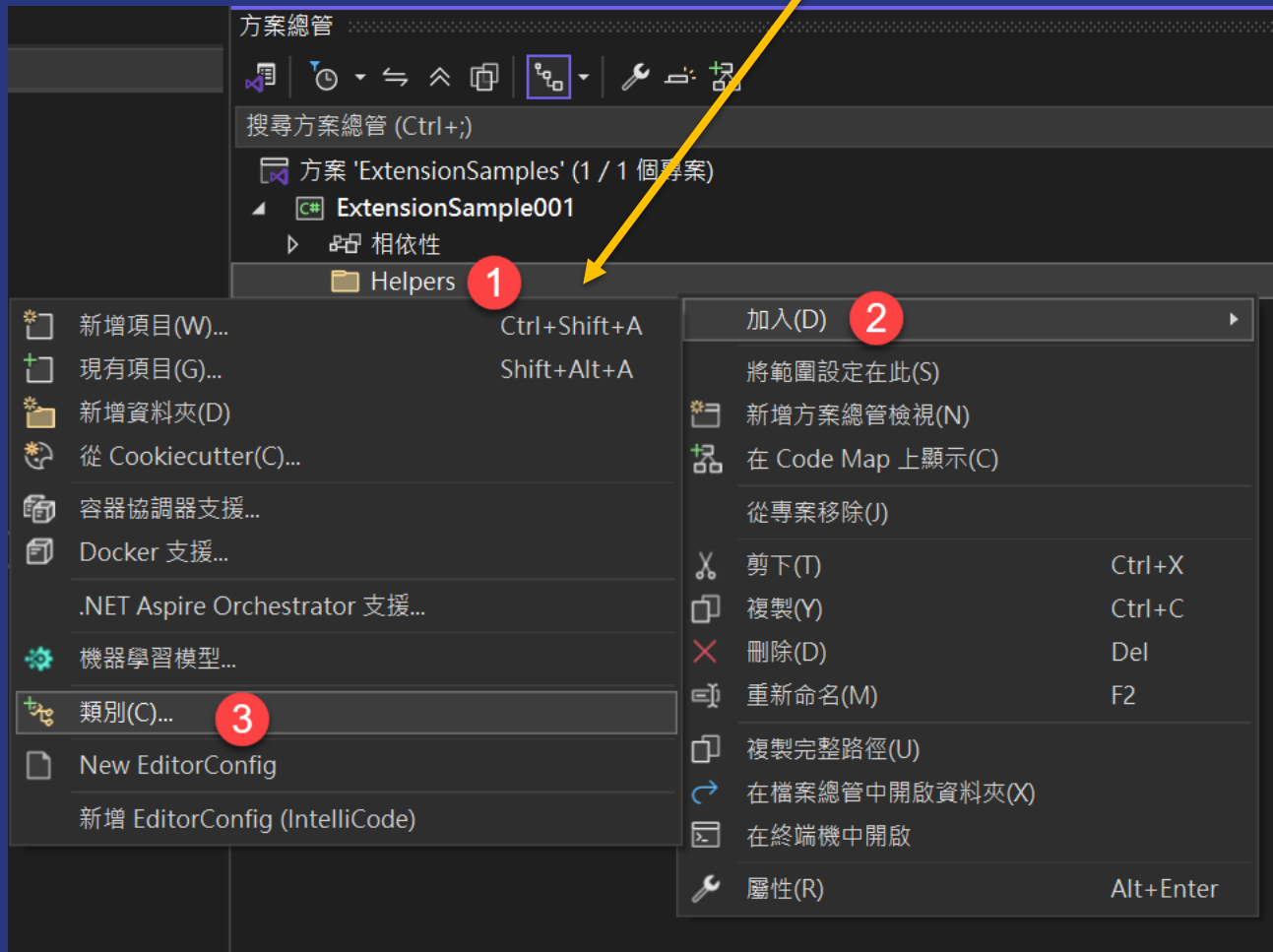
在專案中加入新資料夾 (Helpers)



把那個新資料夾更名為【Helpers】



在【Helpers】加入一個新的類別 命名為 MyClass



你現在會看到長這樣的程式碼

請注意 namespace

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace EextensionSample001.Helpers
{
    class MyClass
    {
    }
}
```

在 Visual Studio 的 C# 開發環境，當你在專案下的子資料夾內建立類別的時候，會自動帶上資料夾的名稱作為 namespace

**** 但是 namespace 是可以自由更改的，不限於和資料夾或專案名稱相同**

修改 MyClass class 的內容 (static class)

我是因為Method宣告太長才折行的，你可以不用折行



```
public static class MyClass
{
    public static List<string> DoWhere(this List<string> source,
                                       Func<string, bool> predicate)
    {
        List<string> result = new List<string>();
        foreach (var item in source)
        {
            if (predicate.Invoke(item))
            {
                result.Add(item);
            }
        }
        return result;
    }
}
```

修改 Main method

```
class Program
{
    static void Main(string[] args)
    {
        List<string> source
            = new List<string> { "Bill", "John", "David", "Tom", "David" };
        var result = source.DoWhere((x) => x == "David");
        // 其實它原來應該是這樣寫
        // var result = MyClass.DoWhere(source, ((x) => x == "David"));
        foreach (var item in result)
        {
            Console.WriteLine(item);
        }
        Console.ReadLine();
    }
}
```

執行

討論

- 擴充方法有一定的重要性，請和你的同學們好好討論它是怎麼設計，又是如何**假裝**成執行個體方法的。



介面型別 Interface type

關於介面型別 (C# 1.0 ~ C# 7.3)

- 介面是一系列方法、屬性、事件與索引的簽章
- 介面是完全抽象的
- 介面的抽象成員不需宣告存取範圍（它一定是 **public**）
- 介面不能定義執行個體欄位與建構式
- C# 不允許在介面中定義靜態成員
- 介面可視為一種契約
- 介面的設計要儘量簡單
- C# 是一種只能繼承自一個類別語言，俗稱單一繼承
- 但是你可以在一個類別上實作多個介面

介面型別的演進

- C# 8.0 允許介面預設實作方法
- C# 11.0 允許介面定義靜態虛擬方法

介面型別的宣告方式

以 interface 關鍵字宣告



```
public interface IShape  
{  
}
```

C# 開發者的慣用介面命名是以大寫 **I** 開頭，
然後接續的名稱首字母也大寫

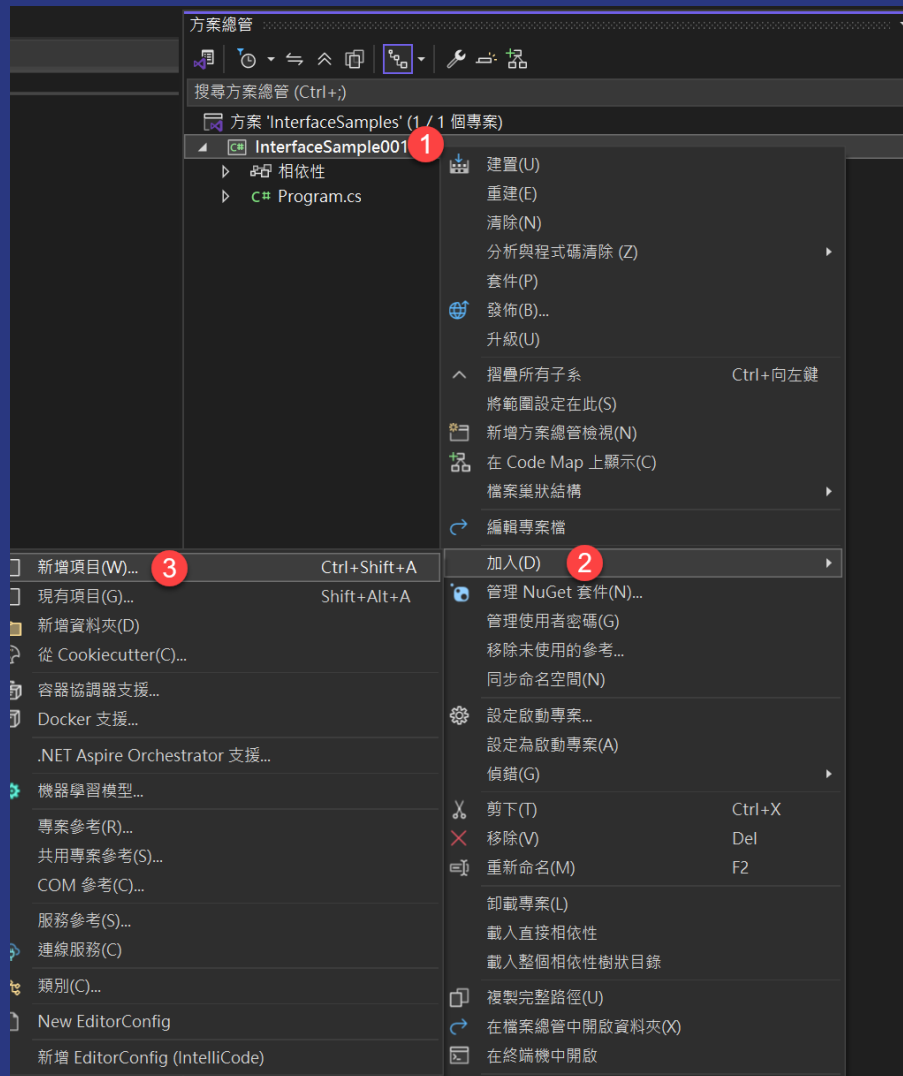
LAB

介面型別的基本練習

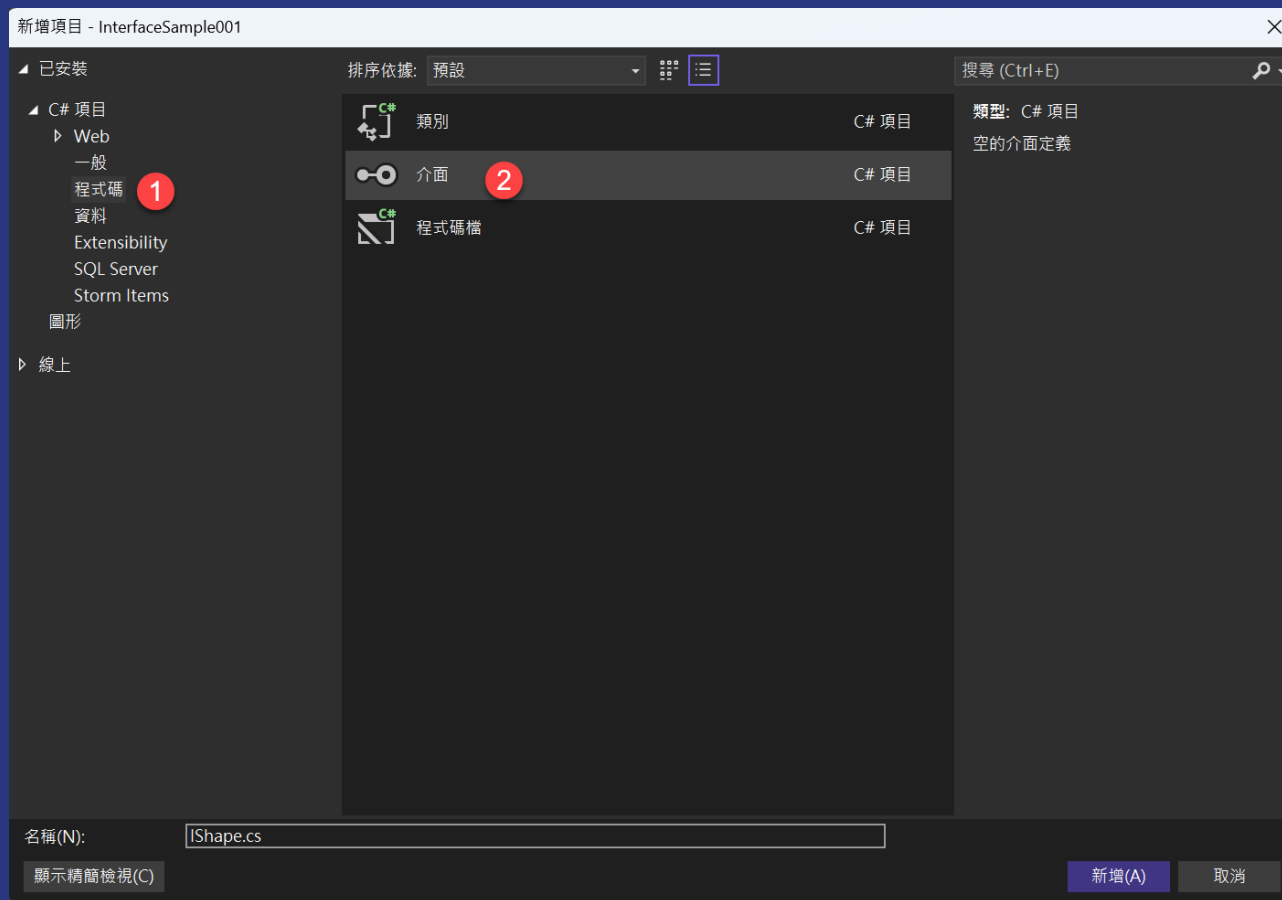
新增一個方案及專案

- 方案名稱：InterfaceSamples
- 專案名稱：InterfaceSample001
- 範本：Console Application

在專案中選擇【加入】→【新增項目】



項目範本使用【介面】(Interface)，命名為 IShape



建立 IShape 介面

```
public interface IShape
{
    double GetArea();
}
```

接著建立一個 MyRectangle class
並宣告其實作 IShape 介面

```
public class MyRectangle : IShape  
{  
  
}
```

和我們之前繼承抽象類別的範例類似，
Visual Studio 會提示你有些宣告在 IShape 介面上的
成員沒有實作。
一樣用燈泡的功能來實作這些成員

0 個參考

```
public class MyRectangle : IShape  
{  
    ...  
}
```



interface InterfaceSample001.IShape

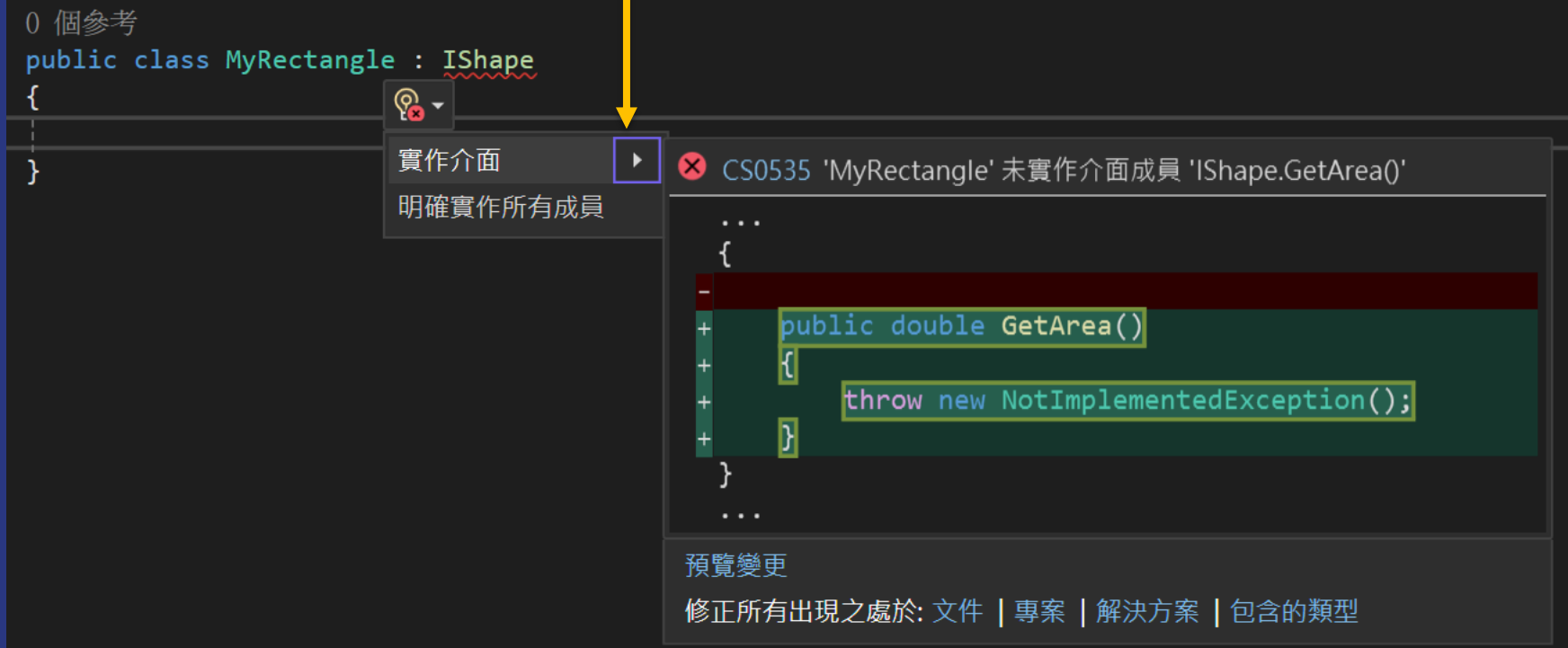
使用 Copilot 加以描述

CS0535: 'MyRectangle' 未實作介面成員 'IShape.GetArea()'

顯示可能的修正 (Alt+Enter 或 Ctrl+.)

請選擇【實作介面】

至於【明確實作介面】等日後你們熟悉了介面的用法再來學。



0 個參考

```
public class MyRectangle : IShape
{
    ...
}
```

實作介面
明確實作所有成員

CS0535 'MyRectangle' 未實作介面成員 'IShape.GetArea()'

```
...
{
    public double GetArea()
    {
        throw new NotImplementedException();
    }
    ...
}
```

預覽變更
修正所有出現之處於: 文件 | 專案 | 解決方案 | 包含的類型

完成 MyRectangle class 的內容

```
public class MyRectangle : IShape
{
    public double Width { get; set; }
    public double Height { get; set; }
    public double GetArea()
    {
        return Width * Height;
    }
}
```

使用一樣的步驟，完成 MyCircle Class

```
public class MyCircle : IShape
{
    public double Radius { get; set; }
    public double GetArea()
    {
        return Math.PI * Math.Pow(Radius, 2);
    }
}
```

完成 Program Class

```
internal class Program
{
    static void Main(string[] args)
    {
        IShape rect = new MyRectangle() { Width = 10, Height = 10 };
        Display(rect);
        IShape circle = new MyCircle() { Radius = 3 };
        Display(circle);
        Console.ReadLine();
    }

    static void Display(IShape shape)
    {
        Console.WriteLine($"這個形狀的面積是 {shape.GetArea()}");
    }
}
```




執行

LAB

介面型別與抽象類別的搭配

開啟 InterfaceSample002

- 先研究 `IParser.cs` 檔案
- 然後是 `PeopleFactory.cs`
- 然後是 `Form1.cs`

類別與介面如何選擇

- 抽象類別的重點在於重用性設計
- 介面設計著重的則是抽象程度
- 重用與彈性 / 血統與能力

討論

- 和你們的夥伴們討論介面的使用方式



隨堂練習

體會介面的作用

- 說明

- `Dictionary<TKey, TValue>` 和 `List<T>` 都可以使用 `Enumerable` class 裡的 `linq` 相關方法，原因是甚麼？

泛型 Generic types

泛型概觀

- .Net Framework 2.0後才出現泛型
- 泛型將型別參數的概念引進 .NET Framework 中，使得類別和方法在設計時，可以先行擱置一個或多個型別規格，直到用戶端程式碼對類別或方法進行宣告或執行個體化時再行處理
- `List<T>`就是一種泛型的設計

泛型可以用在哪裡

- 泛型介面
 - `interface ITest<T>`
- 泛型類別
 - `class Test<T>`
- 泛型方法
 - `void Test<T> (T value)`
 - `T Test<T>()`
- 泛型委派
 - `delegate void Del<T>(T item)`

泛型條件約束

- 定義泛型類別時，可限制用戶端程式碼在執行個體化類別時的型別
- 使用 `where` 內容關鍵字指定條件約束
- `where T: struct`
 - 型別引數必須是實值型別
- `where T : class`
 - 型別引數必須是參考型別

- `where T : new()`
 - 型別引數必須擁有公用的無參數建構函式
 - 將 `new()` 條件約束與其他條件約束一起使用時，一定要將其指定為最後一個
- `where T : <base class name>`
 - 型別引數必須本身是指定的基底類別，或衍生自該類別
- `where T : <interface name>`
 - 型別引數必須本身是指定的介面，或實作該介面
- `where T : U`
 - 提供給 `T` 的型別引數必須是（或衍生自）提供給 `U` 的引數

LAB

泛型宣告

新增一個方案及專案

- 方案名稱：GenericSamples
- 專案名稱：GenericSample001
- 範本：Console Application

加入 Class1 class 並改成泛型

```
public class Class1<T>
{
    public T Data { get; set; }
}
```

完成 Program class

```
class Program
{
    static void Main(string[] args)
    {
        Class1<int> o1 = new Class1<int>();
        string s1 = o1.Data.GetType().ToString();
        Display("o1.Data", s1);

        Class1<bool> o2 = new Class1<bool>();
        string s2 = o2.Data.GetType().ToString();
        Display("o2.Data", s2);

        Console.ReadLine();
    }

    static void Display(string varName, string typeString)
    {
        Console.WriteLine($"{varName} 的型別是 {typeString}");
    }
}
```




執行

LAB

泛型的 DoWhere
ExtensionSample001 改成泛型

在 GenericSamples 加入新專案

- 方案名稱：GenericSamples
- 專案名稱：GenericSample002
- 範本：Console Application

建立 MyClass class 的內容 在這邊 DoWhere<T> 就是一個泛型方法

```
public static class MyClass
{
    public static List<T> DoWhere<T>
        (this List<T> source, Func<T, bool> predicate)
    {
        List<T> result = new List<T>();
        foreach (var item in source)
        {
            if (predicate.Invoke(item))
            {
                result.Add(item);
            }
        }
        return result;
    }
}
```

修改 Main method

```
static void Main(string[] args)
{
    List<string> source
        = new List<string> { "Bill", "John", "David", "Tom", "David" };
    var result = source.DoWhere<string>((x) => x == "David");
    foreach (var item in result)
    {
        Console.WriteLine(item);
    }

    // 現在甚麼型別都可以來一下了
    List<int> source2 = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8 };
    var result2 = source2.DoWhere((x) => x > 3);
    foreach (var item in result2)
    {
        Console.WriteLine(item);
    }

    Console.ReadLine();
}
```

這邊是明確宣告泛型型別

這邊是利用傳入的參數推斷泛型型別



執行

泛型方法中的明確宣告

- 在剛剛的例子，由於可以使用參數推斷泛型型別，因此可以不需要明確地宣告泛型型別。
- 但是，如果一個泛型方法，其泛型型別只用在回傳值的時候，使用該泛型方法就必須明確宣告其泛型型別

討論

- 和你們的夥伴們討論泛型的定義與使用



Attribute

Attribute

- **Attribute** 是一種和一般命令程式不同的設計方式，通常被稱為『宣告式設計』
- 當一個 **Attribute** 被加入到某個元素時，該元素就被認為具有此特性的功能或性質
- **Attribute** 是被動的，無法存取目標物
- 要建立一個可以當作 **Attribute** 的類別，必須繼承 **Attribute** 類別
- 在 .Net Framework 裡已經有的 **Attribute** 常常都是讓 編譯器、Visual Studio 開發環境或是 CLR 所呼叫的。

LAB

體會 Attribute

新增一個方案及專案

- 方案名稱：AttributeSamples
- 專案名稱：AttributeSample001
- 範本：Console Application

這次要用到的 Attribute

- ObsoleteAttribute 類別
 - 這個 Attribute 是讓我們用來標記不再使用的程式項目

加入 Class1 class 並寫下以下的 method

```
public class Class1
{
    public void Execute()
    {

    }

    public void Begin()
    {

    }
}
```

修改 Main method

```
static void Main(string[] args)
{
    Class1 o = new Class1();
    o.Begin();
    o.Execute();
}
```

把游標移到 o.Execute 上，看起來沒啥異狀

```
static void Main(string[] args)
{
    Class1 o = new Class1();
    o.Begin();
    o.Execute();
}
```

 void Class1.Execute()

 使用 Copilot 加以描述

回到 Class1 class 在 Execute 方法上加入 Attribute

```
public class Class1
{
    [Obsolete("就說不要用這個方法")]
    public void Execute()
    {

    }

    public void Begin()
    {

    }
}
```

回到 Main method
把游標移到 o.Execute 上，
就出現了“已取代”的警告了

0 個參考

```
internal class Program
```

```
{
```

0 個參考

```
static void Main(string[] args)
```

```
{
```

```
    Class1 o = new Class1();
```

```
    o.Begin();
```

```
    o.Execute();
```

```
}
```



[已取代] void Class1.Execute()

使用 Copilot 加以描述

CS0618: 'Class1.Execute()' 已經過時: '就說不要用這個方法'

顯示可能的修正 (Alt+Enter 或 Ctrl+.)

例外處理 Exception Handling

例外處理概觀

- C# 語言的例外狀況 (Exception) 處理功能可協助您處理任何在程式執行時所發生的未預期或例外情況。例外狀況處理 (Exception Handling) 會使用 `try`、`catch` 和 `finally` 關鍵字，嘗試執行可能不會成功的動作、適時處理失敗，以及在事後清除資源
- Common Language Runtime (CLR)、.NET Framework 或任何協力廠商程式庫，或是應用程式程式碼都可能產生例外狀況。例外狀況是透過 `throw` 關鍵字建立的。

LAB

為 `int.Parse` 建立例外處理

新增一個方案及專案

- 方案名稱：ExceptionSamples
- 專案名稱：ExceptionSample001
- 範本：Console Application

修改 Program class

```
internal class Program
{
    static void Main(string[] args)
    {
        DoParse();
    }

    static void DoParse()
    {
        try
        {
            string s = "ABC";
            int i = int.Parse(s);
        }
        catch (Exception ex)
        {
            Console.WriteLine($"發生例外 {ex.Message}");
        }
        finally
        {
            Console.WriteLine("執行了 finally 區塊");
        }
    }
}
```




執行

討論

- 清楚 try-catch-finally 的用法了嗎？



物件導向的基本概念

物件的特性

- 物件必須有一個資料結構來存放資料
- 物件有狀態和行為
- 物件要能被識別
- 物件可以被創造及消滅
- 物件有生命週期
- 物件要能為自己負責

類別與物件的關係

- 外界的真實事物都有其相似或相同性質之處
- 類別的形成即是透過分類的過程將一群類似的物件抽象化成一個概念

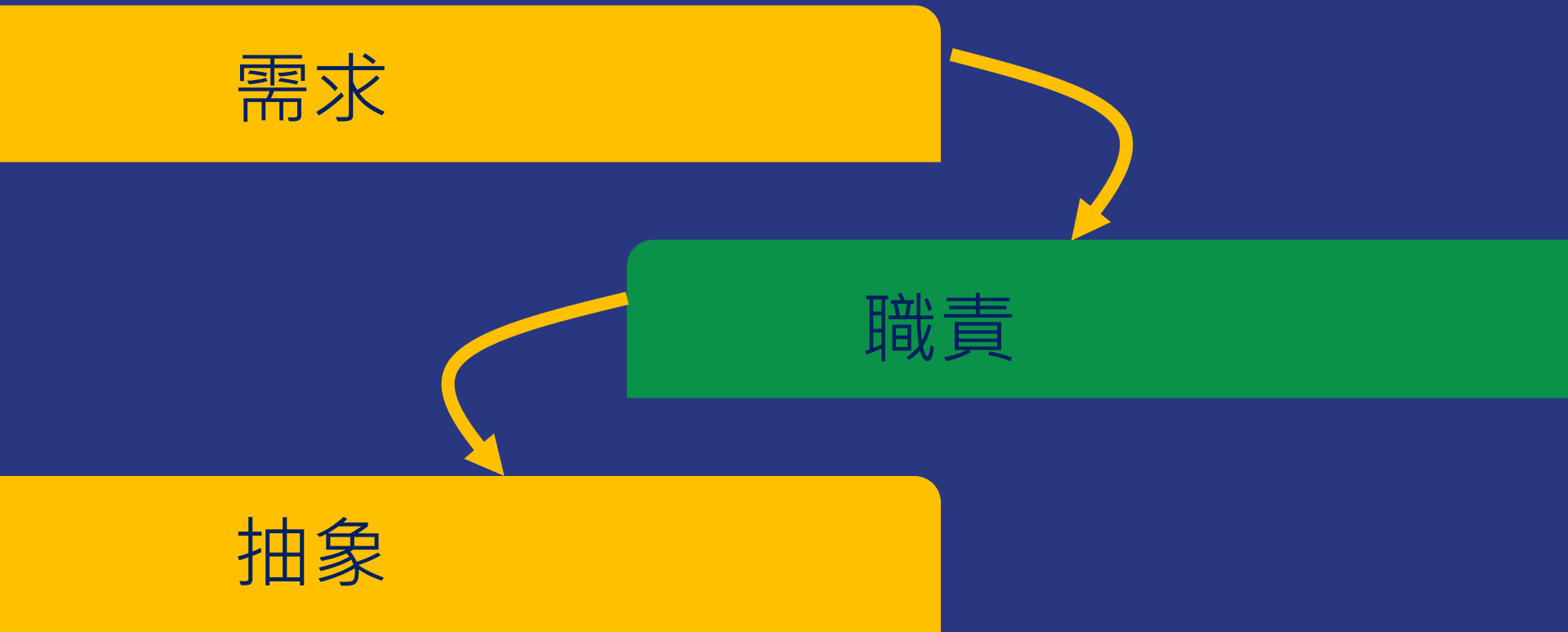
往良好的程式碼邁進

流程

需求

職責

抽象



目標

高內聚

成員內聚 vs 語意內聚
執行個體方法成立要件
(其中一項就可成立):
1. 虛擬方法
2. 成員內聚性

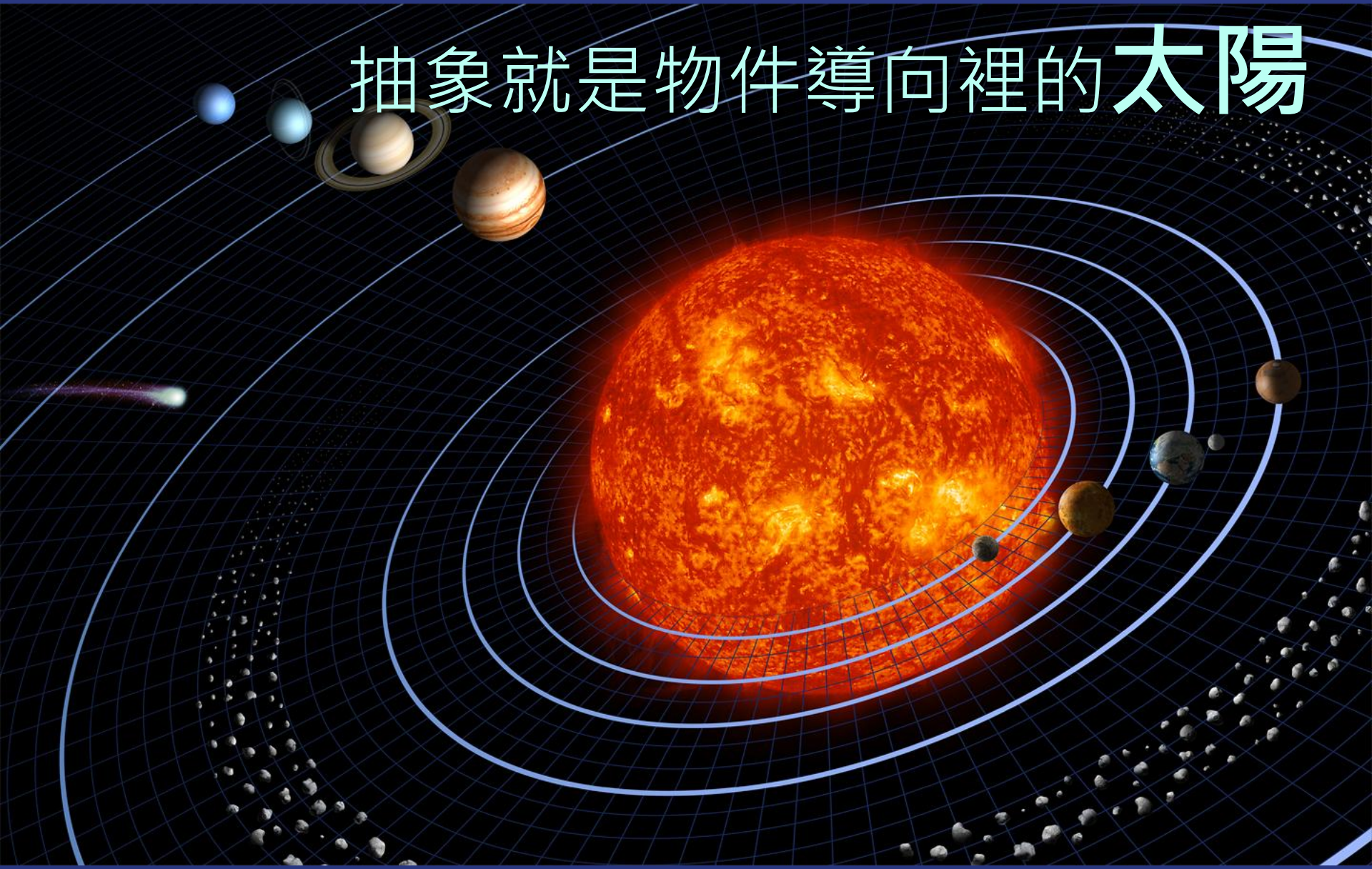
低耦合

農夫過河物件導向版

- 讓狼知道自己要吃羊，羊自己知道要吃菜
- FarmerAcrossRiverSample01_2

抽象

抽象就是物件導向裡的太陽



抽象是...

- 找出關鍵性特徵並加以描述
- 簡化模型以協助思考與運用

抽象能力的重要性

- 分類的基礎技巧就是抽象化
- 分類是必須的，但方式並非絕對的







為什麼要依賴抽象？

演算法的抽象

- 計算 **1~100** 的總和，你會選擇
 1. 用迴圈？
 2. 用數學公式？

物件導向三特性

三個特性

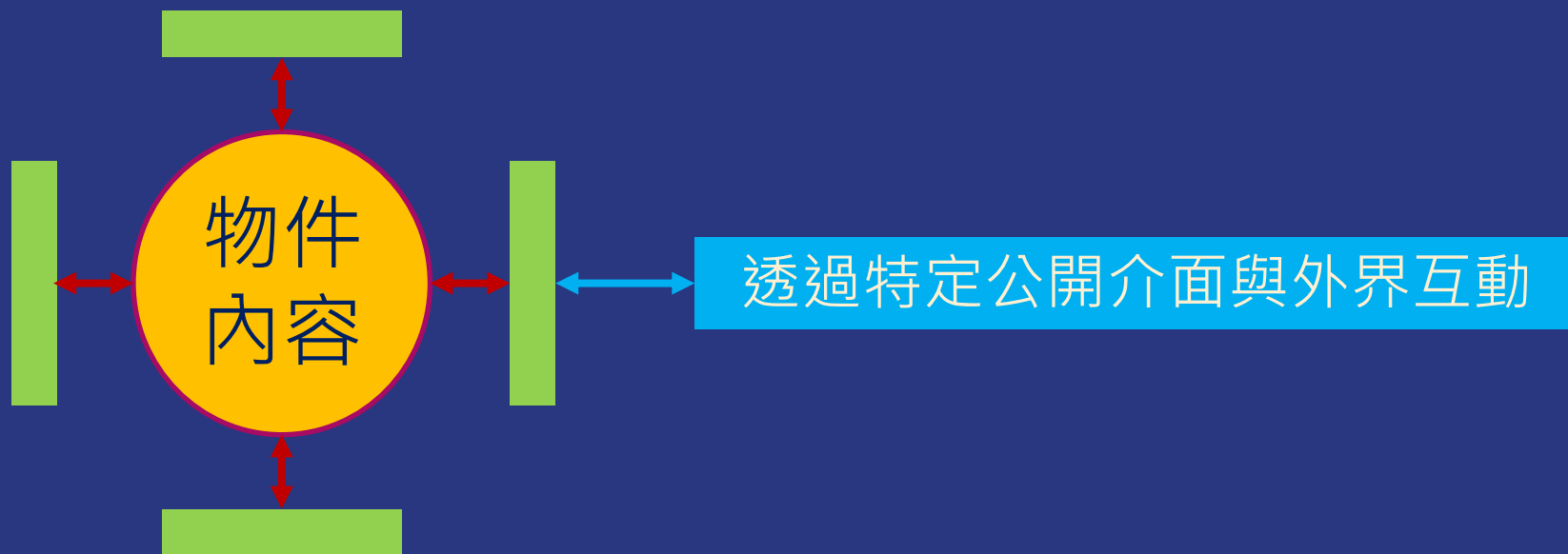
- 繼承
- 封裝
- 多型

繼承

- 繼承者會擁有被繼承者的型別特徵（非靜態）
- C# 中的繼承
 - 繼承一個上層類別（只能一個）
 - 實做介面（可以多個）

封裝

- 隱藏不必要為外界所知的資訊
- 隱藏行為的變化

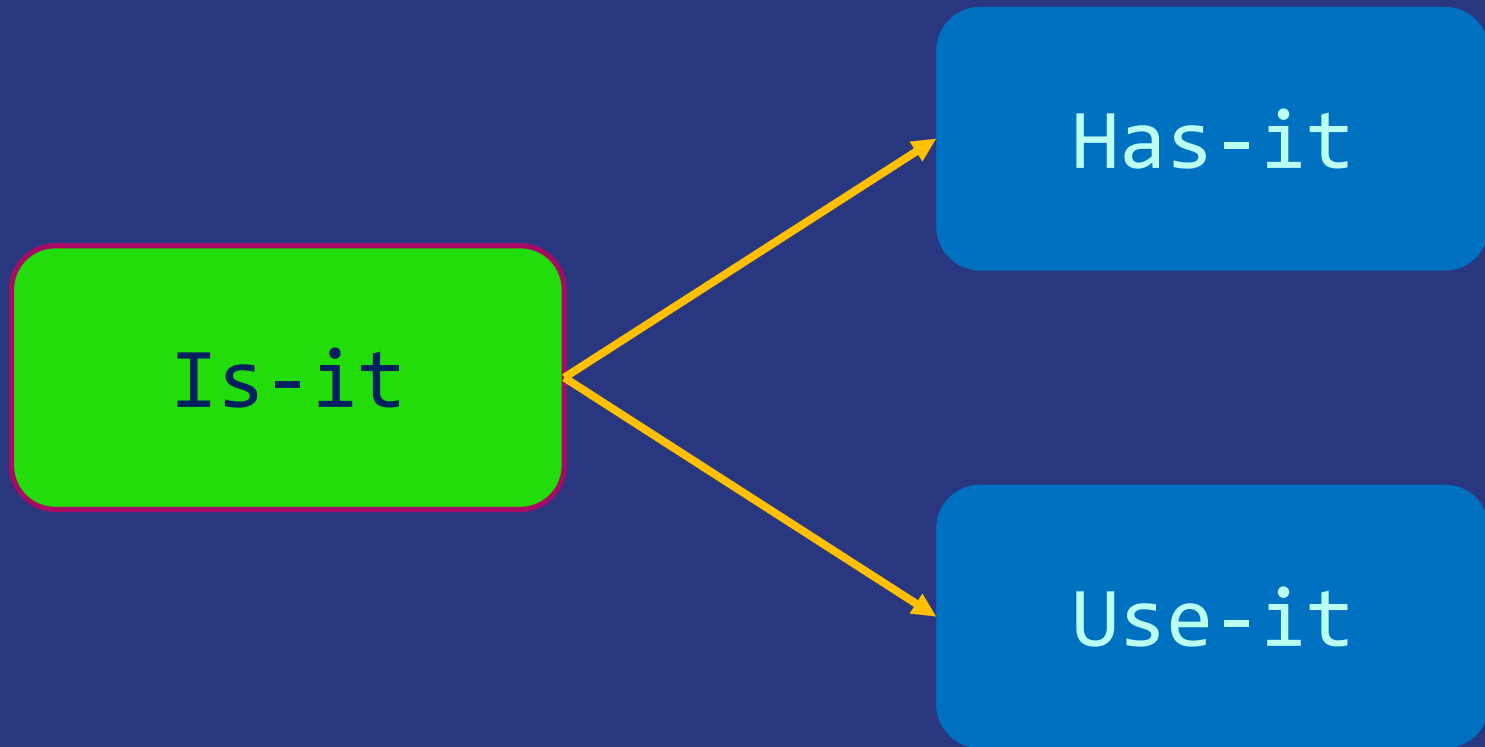


多型

- 廣義多型 (universal polymorphism)
 - 繼承式多型 (inclusion)
 - 參數式多型 (parametric)
- 特設多型 (ad hoc polymorphism)
 - 多載 (overloading)
 - 強制同型 (coercions)

來源: [On Understanding Types, Data Abstraction, and Polymorphism](#)
1.3. Kinds of Polymorphism

繼承 VS 組合/聚合





Build School Announcement

在這裏你將學到

Learn How to Learn

- 學新東西、新技術的能力
- 尋找解答的能力
- 隨時吸取新知識的能力

跟著你一輩子的能力 ...

對學生的好處

不只是就業銜接，培養自我解決問題的能力及信心!



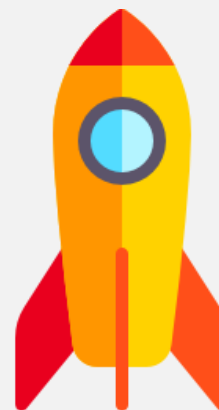
解決問題能力 (Problem-Solving)

- Learn how to learn
- Peer Learning
- Project-based



就業銜接 (Professional skills ready)

- Technical +
Soft Skill



擴散及影響 (Make impacts)

- Mentor
Networks